

데이터 전처리

2026-2 COW AI팀 사전 과제 · 2주차

김효준

2026년 8월 11일

요약

분석 주제: DAICON 갑상선암 진단 분류 데이터 (양성/악성 이진 분류)
결측치 · 이상치 · 인코딩 · Scaling · Feature Engineering 5개 항목을 적용하고, 각 선택을 모델 성능으로 검증

차례

0. 이번 주차의 출발점과 실험 설계	1
1. 결측치 처리	3
2. 이상치 처리	5
3. 인코딩	7
4. Scaling	9
5. Feature Engineering	11
6. 최종 파이프라인 및 검증	13
7. 마무리	15
참고 문헌	16

0. 이번 주차의 출발점과 실험 설계

1주차에서 확인한 것

발견	2주차에 미치는 영향
결측치·중복이 0건	결측치 처리에 적용할 대상이 없음 → 인위적 주입 실험으로 대체
수치형 5개가 모두 균등분포 (첨도 -1.2)	이상치가 존재할 수 없음 → 처리하지 않는 것이 옳음
단변량 신호는 거의 없고 3개의 상호작용 에 구조가 있음	Feature Engineering이 이번 주차의 핵심
악성 12%의 불균형	평가지표는 F1, 분할은 Stratified

실험 설계

모든 비교는 **전처리만 바꾸고 모델은 고정**했다. 그래야 성능 변화의 원인을 전처리로 특정할 수 있다.

항목	설정
모델	로지스틱 회귀(선형), 랜덤 포레스트(트리) — 둘 다 <code>class_weight='balanced'</code>
평가지표	F1 (1주차에서 Accuracy가 부적합함을 확인)
빠른 비교	계층적 80/20 홀드아웃 (검증셋 17,432건, 양성 약 2,092건)
최종 확인	Stratified 5-Fold 교차검증
난수 시드	42 (전 구간 고정)

1. 결측치 처리

방법 설명

방법	설명	장점	단점
삭제(Listwise)	결측이 있는 행을 통째로 제거	왜곡이 없음	데이터 손실이 큼
평균 대체	해당 열의 평균으로 채움	간단, 평균 보존	분산이 줄고 분포 왜곡
중앙값 대체	중앙값으로 채움	이상치에 강함	분산 축소는 동일
최빈값 대체	가장 흔한 값으로 채움	범주형에 적합	수치형에는 부적합
별도 범주 + 플래그	결측을 하나의 값으로 취급하고 결측 여부를 새 변수로 추가	결측 자체가 정보일 때 유리	변수가 늘어남

적용한 이유

이 데이터에는 결측치가 0건이다. 처리할 대상이 없다.

그렇다고 항목을 건너뛰면 각 기법의 특성을 알 수 없으므로, **인위적으로 결측을 주입한 사본**을 만들어 통제된 실험을 수행했다. 이 방식의 장점은 **정답(원본)을 알고 있다**는 것이다. 각 기법이 원본에서 얼마나 벗어나는지 정확히 측정할 수 있다.

적용 방법

수치형 2개(Age, TSH_Result)와 범주형 1개(Race)에 각각 **10%씩 무작위(MCAR) 결측을 주입**한 뒤 5가지 기법을 적용했다.

적용 결과

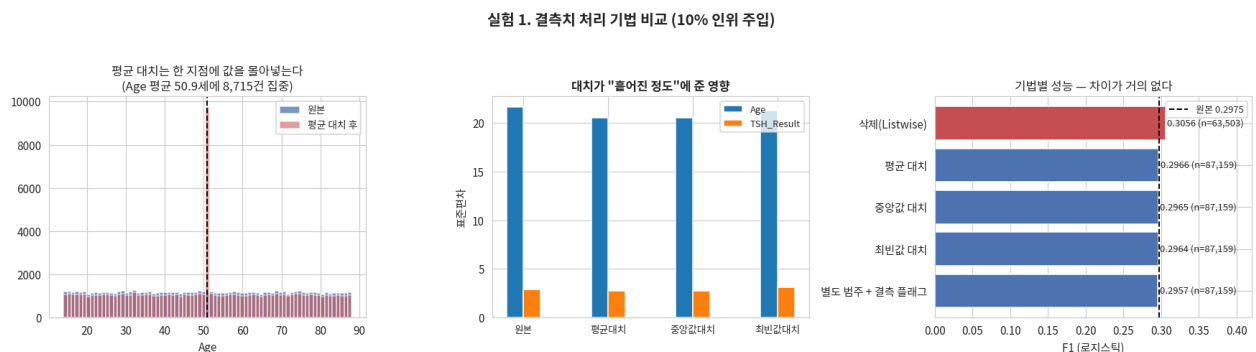


그림 1: 실험 1. 결측치 처리 기법 비교

기법	행수	손실	Age 표준편차	TSH 표준편차	F1(로지스틱)
삭제(Listwise)	63,503	23,656	21.625	2.861	0.3056
평균 대체	87,159	0	20.521	2.715	0.2966
중앙값 대체	87,159	0	20.521	2.715	0.2965
최빈값 대체	87,159	0	21.285	3.094	0.2964

기법	행수	손실	Age 표준편차	TSH 표준편차	F1(로지스틱)
별도 범주 + 플래그	87,159	0	20.521	2.715	0.2957
[원본 — 결측 없음]	87,159	0	21.639	2.861	0.2975

(1) 삭제법은 27%를 잃는다. 각 열의 결측률은 10%인데 행 기준 손실은 27.14%다. 결측이 있는 열이 늘어날수록 손실이 급격히 커진다.

(2) 평균·중앙값 대치는 분포를 좁힌다. Age 표준편차가 21.639 → 20.521로 약 5.2% 축소됐다. 같은 값을 8,715건이나 한 지점에 몰아넣었기 때문이다. “평균은 보존되지만 분산은 과소평가된다”는 대체법의 알려진 한계가 그대로 나타났다.

(3) 최빈값 대치는 수치형에 부적절하다. TSH_Result 표준편차가 2.861 → 3.094로 오히려 증가했다. 최빈값이 분포의 중심이 아닌 곳에 있어 값이 엉뚱한 지점에 쌓였기 때문이다.

(4) 그런데 F1은 거의 변하지 않았다 (0.2957 ~ 0.3056). 이는 대체법이 훌륭해서가 아니라, 결측을 주입한 Age와 TSH_Result가 1주차에서 이미 “타겟과 무관”으로 판명된 변수이기 때문이다. 예측에 기여하지 않는 변수는 어떻게 채워도 성능이 변하지 않는다.

얻은 교훈: 결측치 처리 기법의 선택은 그 변수가 **예측에 기여할 때만** 의미가 있다. “평균 대체가 표준”이라고 기계적으로 적용하기보다 해당 변수의 중요도를 먼저 확인해야 한다.

최종 결정

실제 데이터에는 결측이 없으므로 아무 처리도 적용하지 않는다. 향후 결측 발생 시의 원칙은 다음과 같이 정했다.

- 수치형 → 중앙값 대체 (분산 축소 정도는 평균과 같지만 이상치에 더 강함)
- 범주형 → 별도 범주(MISSING)로 처리
- 결측률 30% 초과 열 → 열 제거 검토
- 삭제법은 사용하지 않는다 — 손실이 너무 크다

2. 이상치 처리

방법 설명

기준	판정 방법
IQR	$Q1 - 1.5 \times IQR$ 미만 또는 $Q3 + 1.5 \times IQR$ 초과
Z-score	평균에서 몇 표준편차 떨어졌는가, 보통 $ z > 3$
백분위	하위 1% 미만, 상위 1% 초과

처리 방법은 **제거**(행 삭제), **클리핑**(경계값으로 자르기), **변환**(로그 등)이 있다.

적용한 이유

1주차에서 IQR 기준 이상치가 0건임을 확인했다. 이번에는 **다른 두 기준으로도 교차 확인**하고, 관행대로 처리를 강행하면 어떤 손해가 있는지 정량적으로 측정했다.

적용 결과

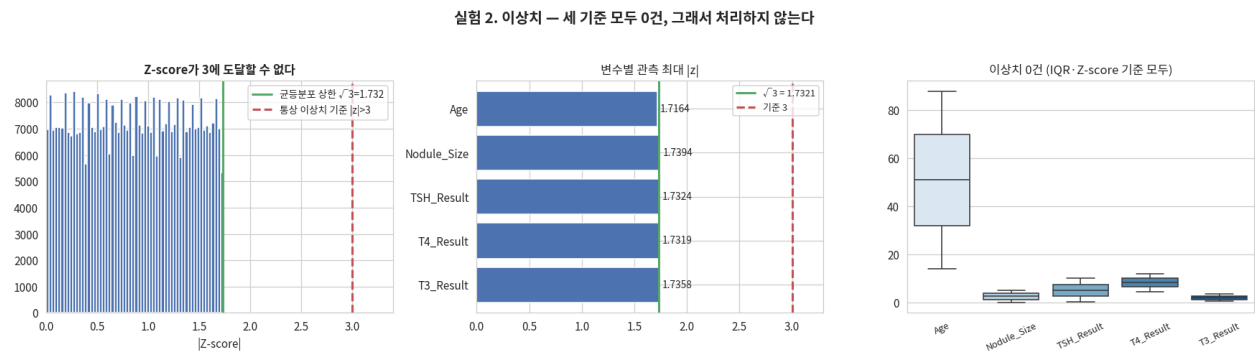


그림 2: 실험 2. 이상치 — 세 기준 모두 0건

변수	실제 범위	IQR 이상치	최대 $ z $	Z 이상치
Age	14 ~ 88	0	1.7164	0
Nodule_Size	0.00 ~ 5.00	0	1.7394	0
TSH_Result	0.10 ~ 10.00	0	1.7324	0
T4_Result	4.50 ~ 12.00	0	1.7319	0
T3_Result	0.50 ~ 3.50	0	1.7358	0

Z-score로는 이 데이터에서 이상치를 찾을 수 없다 — 수학적 증명

관측된 최대 $|z|$ 가 **1.7164 ~ 1.7394**로 기준값 3에 한참 못 미친다. 이것은 우연이 아니다.

구간 $[a, b]$ 의 균등분포에서 평균은 $\mu = \frac{a+b}{2}$, 표준편차는 $\sigma = \frac{b-a}{\sqrt{12}}$ 이므로, 최댓값 b 의 Z-score는

$$z_{\max} = \frac{b - \mu}{\sigma} = \frac{(b - a)/2}{(b - a)/\sqrt{12}} = \frac{\sqrt{12}}{2} = \sqrt{3} \approx 1.7321$$

균등분포에서 $|z|$ 는 $\sqrt{3}$ 을 넘을 수 없다. 관측값이 이 이론값과 소수점 둘째 자리까지 일치했다. 따라서 $|z| > 3$ 기준을 아무리 적용해도 이상치는 영원히 0건이다.

그래도 클리핑을 강행하면

항목	결과
값이 바뀐 셀	6,736 개
영향받은 행	6,543 건 (7.51%)
F1 변화	0.2975 → 0.2978 (+0.0002)

멀쩡한 데이터를 7.51% 훼손하면서 얻는 것이 없다.

최종 결정

이상치 처리를 적용하지 않는다. “처리하지 않기로 결정했다”는 것도, 세 기준으로 검증하고 강행 시 손해까지 측정했다면 근거 있는 전처리 결정이다.

3. 인코딩

방법 설명

머신러닝 모델은 숫자만 다루므로 Race = 'ASN' 같은 문자열을 숫자로 바꿔야 한다.

방식	방법	특징
Label Encoding	ASN=0, AFR=1, CAU=2 ...	열이 안 늘어나지만 없는 순서 관계를 만들
One-Hot Encoding	값마다 0/1 열을 하나씩	순서 왜곡 없음, 열이 늘어남
Binary Encoding	값이 2개일 때 0/1로	One-Hot과 정보량 동일하면서 열 절약
Target Encoding	각 범주의 타겟 평균값으로	강력하지만 데이터 누수 위험

적용한 이유

범주형 9개를 숫자로 바꾸지 않으면 학습 자체가 불가능하다. 1주차 계획에서 **이진 7개는 Binary, 다범주 2개 (Country, Race)는 One-Hot**을 쓰기로 했는데, 이 선택이 옳은지 네 가지를 모두 만들어 비교했다.

적용 결과

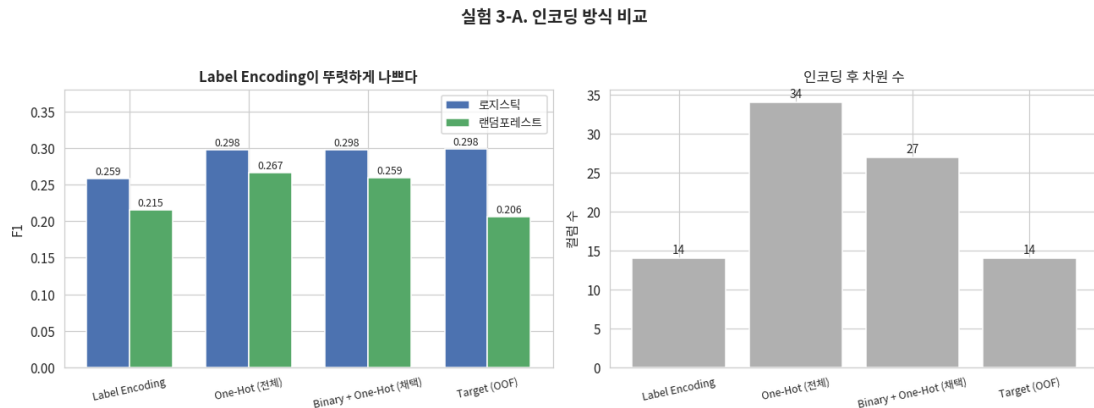


그림 3: 실험 3-A. 인코딩 방식 비교

인코딩	컬럼수	F1(로지스틱)	F1(랜덤포레스트)
Label Encoding	14	0.2587	0.2150
One-Hot (전체)	34	0.2977	0.2665
Binary + One-Hot (채택)	27	0.2975	0.2591
Target (OOF)	14	0.2983	0.2061

Label Encoding이 명확히 나쁘다. Country를 IND=0, RUS=1, KOR=2 ...로 바꾸면 모델은 “KOR은 IND보다 2만큼 크다”고 이해한다. 존재하지 않는 순서와 간격을 지어내는 것이다. 선형 모델은 이 가짜 순서를 그대로 믿어 F1이 0.0388 하락했고, 트리 모델도 특정 국가 하나를 분리하려면 여러 번 분할해야 해서 0.0441 하락했다.

데이터 누수(Leakage) 검증

Target Encoding은 타깃 값으로 특징을 만들기 때문에 **학습 시점에 알 수 없어야 할 정보가 섞여 들어갈 위험**이 있다.

(A) 이 데이터의 실제 변수로는 누수가 나타나지 않았다.

방식	학습셋 F1	검증셋 F1	격차
잘못된 방식(전체 평균)	0.3037	0.2992	+0.0045
올바른 방식(OOF)	0.3026	0.2983	+0.0043

격차가 거의 같다. 범주가 2~10개뿐이고 표본이 87,159건이라 어떻게 나뉘어도 범주별 평균이 거의 변하지 않기 때문이다.

(B) 고유값이 많은 변수라면 결과가 완전히 달라진다.

타깃과 아무 관계 없는 **순수 무작위 변수(고유값 3,000개)**를 만들어 넣었다. 이 변수는 정보가 0이므로 **성능이 떨어지는 것이 정상**이다.

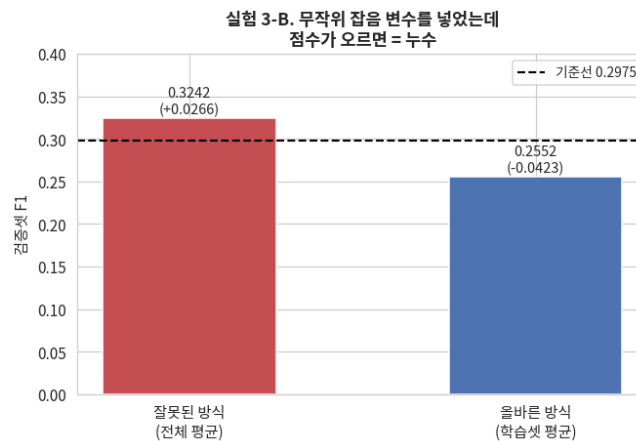


그림 4: 실험 3-B. 데이터 누수 시연

방식	학습셋 F1	검증셋 F1	기준선(0.2975) 대비	판정
잘못된 방식 (전체 데이터 평균)	0.3325	0.3242	+0.0266	성능이 오른 것처럼 보임 → 착시
올바른 방식 (학습셋 평균만)	0.3406	0.2552	-0.0423	성능이 떨어짐 → 정직한 신호

쓸모없는 변수를 넣었는데 검증 점수가 올랐다면, 그것이 바로 누수의 증거다. 검증 점수가 이유 없이 오르면 기뻐할 게 아니라 누수를 의심해야 한다.

최종 결정

Binary(이진 7개) + One-Hot(Country, Race)를 채택한다.

- One-Hot 전체(34열) 대비 로지스틱 F1 차이가 0.0002로 무시할 수준이면서 **컬럼 7개를 절약**한다
- Label Encoding은 가짜 순서를 만들어 명확히 해롭다
- Target Encoding은 로지스틱에서 근소하게 높지만 랜덤포레스트에서 최악(0.2061)이고 누수 관리 비용이 크다

4. Scaling

방법 설명

방법	변환식	결과 범위
StandardScaler	$(x - \mu) / \sigma$	평균 0, 표준편차 1
MinMaxScaler	$(x - \min) / (\max - \min)$	0 ~ 1

적용한 이유

변수마다 범위가 다르면 범위가 넓은 변수가 계산을 지배한다. 이 데이터에서 Age(폭 74)는 T3_Result(폭 3.0)보다 25배 넓다.

1주차에서 “이 변수들이 균등분포이므로 정규성을 가정하는 Standard보다 MinMax가 적합하다”고 판단했다. 이 주장을 세 종류의 모델로 검증했다.

적용 결과

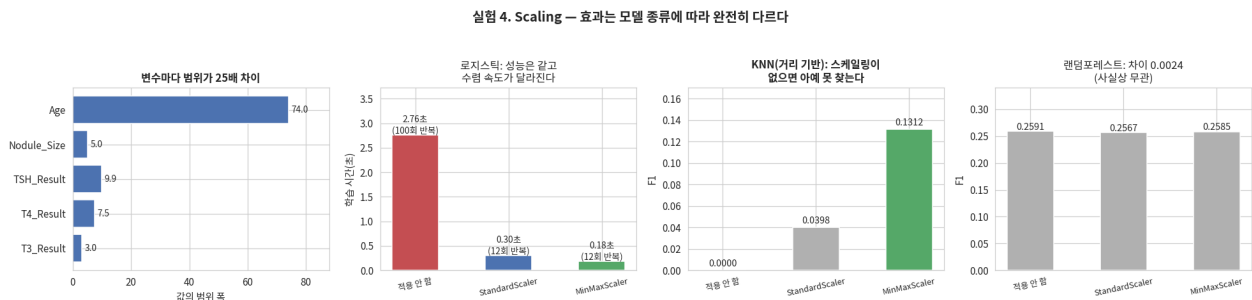


그림 5: 실험 4. Scaling — 효과는 모델 종류에 따라 완전히 다르다

(1) 로지스틱 회귀 — 성능은 같고, 수렴이 빨라진다

스케일링	F1	학습시간	반복횟수
적용 안 함	0.2975	1.75초	100회
StandardScaler	0.2975	0.15초	12회
MinMaxScaler	0.2977	0.16초	12회

F1은 사실상 동일하지만 학습 시간이 약 11배 단축되고 반복 횟수가 100회 → 12회로 줄었다. 스케일링의 본질적 효과는 성능이 아니라 최적화의 안정성과 속도다.

(2) KNN — 스케일링 없이는 작동 자체를 못 한다

스케일링	F1
적용 안 함	0.0000
StandardScaler	0.0398

스케일링	F1
MinMaxScaler	0.1312

스케일링을 안 하면 F1이 **정확히 0**이다. Age가 거리 계산을 완전히 지배해 양성을 단 한 건도 찾지 못한다.

주목할 점은 **MinMax가 Standard보다 3.3배 좋다**는 것이다. Standard는 데이터를 $-1.73 \sim +1.73$ 범위로 보내지만 변수마다 그 폭이 미묘하게 다르다. 반면 MinMax는 **모든 변수를 정확히 [0, 1]로 통일**해 거리 계산에서 완벽히 동등한 가중치를 준다. 1주차의 “균등분포에는 MinMax가 적합하다”는 판단이 실측으로 확인됐다.

(3) 랜덤 포레스트 — 무관하다

스케일링	F1
적용 안 함	0.2591
StandardScaler	0.2567
MinMaxScaler	0.2585

세 값의 최대 차이가 **0.0024**로 무작위 변동 수준이다. 트리는 “값이 몇인가”가 아니라 “**이 값보다 큰가 작은가**”로만 분할하므로, 단조 변환인 스케일링은 분할 순서를 바꾸지 않는다.

(4) 스케일링은 분포 모양을 바꾸지 않는다

스케일러	최소	최대	왜도(평균)	첨도(평균)
적용 안 함	0.0000	88.0000	0.0001	-1.2004
StandardScaler	-1.7394	1.7319	0.0001	-1.2004
MinMaxScaler	0.0000	1.0000	0.0001	-1.2004

왜도와 첨도가 완전히 동일하다. 바뀌는 것은 **축의 눈금**뿐이다.

최종 결정

MinMaxScaler를 수치형 5개에만 적용한다. 트리 계열 모델에는 적용하지 않는다.

5. Feature Engineering

방법 설명

기존 변수를 조합하거나 변형해 **모델이 학습하기 쉬운 새 변수를 만드는 작업**이다. 과제에서는 선택 항목이지만 이 데이터에서는 성능의 거의 전부를 좌우했다.

적용한 이유

1주차 EDA에서 이 데이터의 신호가 개별 변수가 아니라 **세 개의 조합에 있음**을 발견했다. 문제는 **로지스틱 회귀 같은 선형 모델이 이런 AND 조건을 원리적으로 표현할 수 없다**는 것이다. 선형 모델은 각 변수의 효과를 더하기만 할 뿐 “둘 다 참일 때만”이라는 곱셈 관계를 만들지 못한다.

적용 방법

```
df['ASN_Family'] = (df.Race == 'ASN') & (df.Family_Background == 'Positive')
df['AFR_Radiation'] = (df.Race == 'AFR') & (df.Radiation_History == 'Exposed')
df['IND_Iodine'] = (df.Country == 'IND') & (df.Iodine_Deficiency == 'Deficient')
df['n_risk'] = ASN_Family + AFR_Radiation + IND_Iodine
df['is_high_risk'] = (df.n_risk >= 1)
```

추가로 수치형 파생 3개(T3/T4 비율, TSH × Nodule_Size, Age 구간화)도 시도했다.

적용 결과

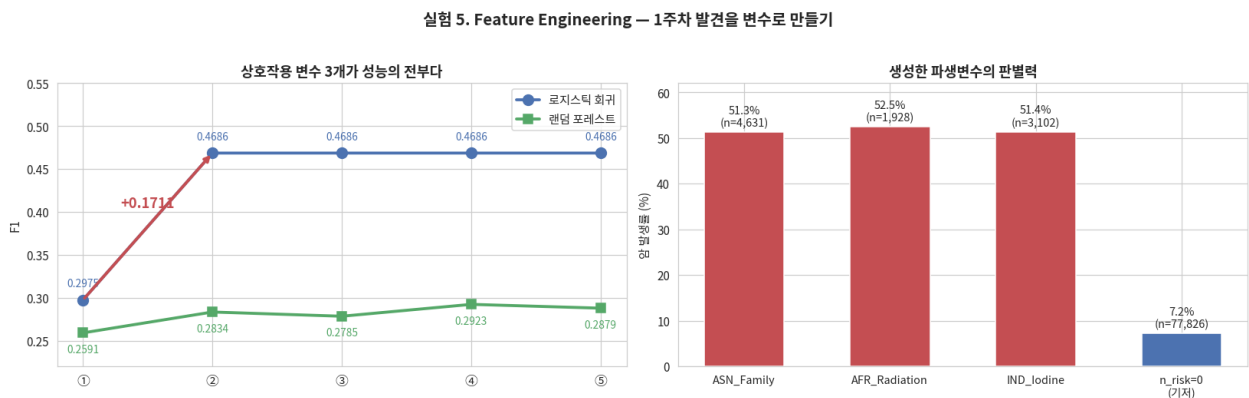


그림 6: 실험 5. Feature Engineering

단계	컬럼수	F1(로지스틱)	변화	F1(랜덤포레스트)
① 기본 인코딩만 (기준선)	27	0.2975	—	0.2591
② + 상호작용 플래그 3개	30	0.4686	+0.1711	0.2834
③ + 위험요인 개수	31	0.4686	+0.0000	0.2785
④ + 위험군 플래그	32	0.4686	+0.0000	0.2923
⑤ + 수치형 파생 3개	35	0.4686	+0.0000	0.2879

(1) 상호작용 변수 3개가 성능의 전부다. 3개 열을 추가했을 뿐인데 로지스틱 F1이 0.2975 → 0.4686으로

+0.1711(상대적으로 57.5% 향상) 올랐다. 이번 주차에서 나온 유일하게 의미 있는 성능 변화다.

생성한 파생변수의 판별력은 다음과 같다.

변수	해당 건수	전체 대비	암 발생률
ASN_Family	4,631	5.31%	51.31%
AFR_Radiation	1,928	2.21%	52.49%
IND_Iodine	3,102	3.56%	51.39%
n_risk = 0 (기저)	77,826	89.29%	7.25%

(2) ③④⑤ 단계는 로지스틱에 아무 정보도 주지 못했다 (0.4686에서 소수점 넷째 자리까지 완전히 동일). n_risk는 세 플래그의 **합**이고 is_high_risk는 그 합이 1인 경우이다. 즉 **이미 가진 변수들의 선형 결합**이므로, 선형 모델은 계수 조정으로 스스로 만들 수 있는 것을 새 변수로 받아도 얻는 것이 없다.

이건 실패가 아니라 **선형 모델의 성질을 확인한 것**이다. 트리 모델에서는 n_risk 하나로 단일 분할이 가능해지므로 여전히 유용할 수 있다(④에서 RF가 0.2785 → 0.2923 상승).

(3) 수치형 파생은 예상대로 효과가 없었다. 랜덤포레스트에서 0.2923 → 0.2879로 오히려 소폭 하락했다. 1주차에서 원 변수의 상관이 $|r| < 0.01$ 이었으므로 예견된 결과다. **그래도 검증했다는 사실 자체가 중요하다.**

(4) 저신호 변수 4개 제거는 효과가 없었다.

설정	컬럼수	F1(로지스틱)	F1(랜덤포레스트)
유지	35	0.4686	0.2879
제거	31	0.4686	0.2800

1주차에서 “실험으로 결정하겠다”고 했던 항목인데, 로지스틱은 완전히 동일하고 랜덤포레스트는 오히려 하락했다. **제거할 이유가 없으므로 유지한다.**

최종 결정

상호작용 플래그 3개, n_risk, is_high_risk를 채택한다. 수치형 파생 3개는 트리 모델에서 소폭 마이너스이므로 **최종 파이프라인에서 제외한다.**

6. 최종 파이프라인 및 검증

파이프라인 구조

학습셋에서 얻은 통계량(스케일러)을 평가셋에 그대로 적용하는 구조로 클래스화했다.

순서	처리
1	이진 범주 7개 → 0/1 매핑
2	Country, Race → One-Hot
3	상호작용 플래그 3개 생성
4	n_risk (위험요인 개수)
5	is_high_risk (위험군 여부)
6	수치형 5개 → MinMaxScaler (학습셋으로 fit, 평가셋에는 transform만)

결과: train 87,159 × 32열, test 46,204 × 32열 (컬럼 구성 일치, 결측 0건)

Stratified 5-Fold 교차검증 결과

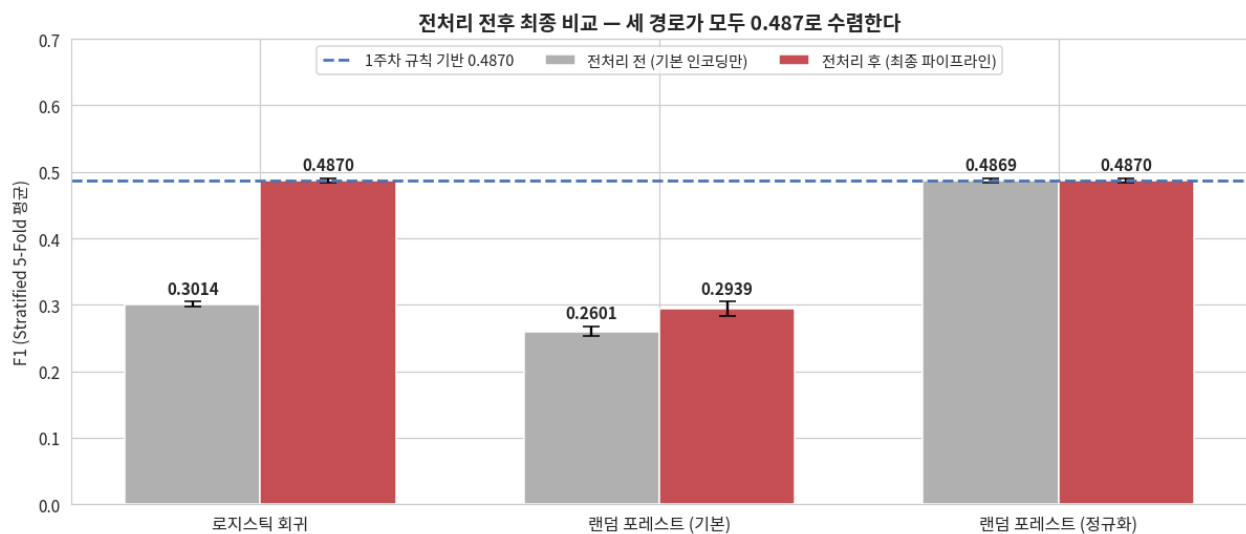


그림 7: 전처리 전후 최종 비교

설정	모델	F1 (5-Fold 평균 ± 표준편차)
전처리 전	로지스틱 회귀	0.3014 ± 0.0035
전처리 후	로지스틱 회귀	0.4870 ± 0.0035
전처리 전	랜덤 포레스트 (기본)	0.2601 ± 0.0075
전처리 후	랜덤 포레스트 (기본)	0.2939 ± 0.0107
전처리 전	랜덤 포레스트 (정규화)	0.4869 ± 0.0036
전처리 후	랜덤 포레스트 (정규화)	0.4870 ± 0.0035

(1) 전처리로 로지스틱 회귀가 +0.1856 상승

0.3014 → 0.4870, 상대적으로 **61.6% 향상**이다. 폴드 간 표준편차가 0.0035로 매우 작아 우연이 아니다.

(2) 서로 다른 세 경로가 모두 0.487에 도달 — 구조적 상한의 증거

접근	F1
1주차 · 조건문 3줄 (학습 없음)	0.4870
2주차 · 로지스틱 회귀 + Feature Engineering	0.4870
2주차 · 정규화 랜덤 포레스트 (FE 없이)	0.4869

완전히 다른 세 방법이 소수점 넷째 자리까지 같은 값에 수렴했다. 이는 우연이 아니라 **이 데이터가 담고 있는 정보의 총량이 정확히 여기까지**라는 뜻이다. 1주차에서 “전체 양성의 53.9%는 예측 불가능한 잡음”이라고 추정했던 것이 실측으로 확인됐다.

(3) Feature Engineering의 진짜 역할

정규화한 랜덤 포레스트는 **파생변수 없이 원본 인코딩만으로 0.4869**를 냈다. 트리는 조건 분기를 중첩할 수 있어 상호작용을 스스로 발견하기 때문이다.

즉 Feature Engineering은 **새로운 정보를 만들어낸 것이 아니라, 이미 데이터에 있던 구조를 선형 모델이 접근할 수 있는 형태로 번역한 것**이다.

모델	상호작용에 도달하는 방법
로지스틱 회귀 (선형)	AND 조건을 표현할 수 없음 → 사람이 만들어 줘야 함
랜덤 포레스트 (트리)	분기를 중첩해 스스로 발견 → 단, 과적합을 막는 정규화가 필요

(4) 기본 랜덤 포레스트가 나뉘었던 이유

깊이 제한이 없으면 89.3%를 차지하는 기저 그룹의 순수한 잡음까지 외운다. max_depth=16, min_samples_leaf=10을 주는 것만으로 **0.2601 → 0.4869 (+0.2268)**로 뛰었다. 3주차 하이퍼파라미터 튜닝의 출발점이 여기서 정해졌다.

7. 마무리

전처리 항목별 요약

#	항목	적용	결과와 근거
1	결측치	미적용	실제 0건. 인위적 주입으로 5개 기법 비교, 향후 원칙 수립
2	이상치	미적용	IQR·Z-score 모두 0건. 균등분포에서 $ z \leq \sqrt{3}$ 임을 증명. 강제 클리핑 시 7.51% 훼손, F1 +0.0002
3	인코딩	적용	Binary + One-Hot. Label은 가짜 순서로 F1 -0.0388, Target은 누수 위험
4	Scaling	적용	MinMaxScaler(수치형 5개). 로지스틱 학습 11배 단축, KNN 0.0000 → 0.1312
5	Feature Eng.	적용	상호작용 3개 + n_risk + is_high_risk. F1 +0.1711

이번 주차에서 배운 것

전처리는 정보를 늘리는 작업이 아니라, 이미 있는 정보를 모델이 쓸 수 있는 형태로 바꾸는 작업이다.

- 결측치·이상치 처리는 적용 대상이 없었고, 억지로 적용하면 손해만 봤다
- 인코딩과 스케일링은 “성능을 올린다”기보다 **모델이 오작동하지 않게 하는 최소 조건**이었다 (KNN F1 0.0000 이 그 증거)
- Feature Engineering만이 유일하게 큰 성능 변화를 만들었는데, 그마저도 **정규화한 트리 모델은 없이도 같은 지점에 도달했다**

3주차로 넘기는 과제

1. **하이퍼파라미터 튜닝** — max_depth=16, min_samples_leaf=10이라는 임의의 값만으로도 +0.2268을 얻었다. 체계적인 탐색이 필요하다
2. **임계값(threshold) 조정** — 기본 0.5가 F1 최적이지 아닐 수 있다. Precision-Recall 곡선에서 최대 지점을 찾는다
3. **성능 상한 인식** — 세 가지 독립적인 접근이 모두 0.487에 수렴했다. 이 벽을 크게 넘는 결과가 나온다면 **먼저 데이터 누수를 의심**해야 한다

참고 문헌

1. scikit-learn developers. “Preprocessing data — User Guide.” <https://scikit-learn.org/stable/modules/preprocessing.html>
2. scikit-learn developers. “Imputation of missing values.” <https://scikit-learn.org/stable/modules/impute.html>
3. scikit-learn developers. “Importance of Feature Scaling.” https://scikit-learn.org/stable/auto_examples/preprocessing/plot_scaling_importance.html
4. Little, R. J. A., & Rubin, D. B. (2019). Statistical Analysis with Missing Data (3rd ed.). Wiley.
5. Micci-Barreca, D. (2001). “A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems.” ACM SIGKDD Explorations, 3(1), 27–32. <https://dl.acm.org/doi/10.1145/507533.507538>
6. Kaufman, S., Rosset, S., & Perlich, C. (2012). “Leakage in Data Mining: Formulation, Detection, and Avoidance.” ACM TKDD, 6(4). <https://dl.acm.org/doi/10.1145/2382577.2382579>
7. Zheng, A., & Casari, A. (2018). Feature Engineering for Machine Learning. O'Reilly Media.
8. Kuhn, M., & Johnson, K. (2019). Feature Engineering and Selection: A Practical Approach for Predictive Models. CRC Press. <http://www.feat.engineering/>
9. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer.
10. DAICON. “갑상선암 진단 분류 해커톤 : 양성과 악성, AI로 정확히 구분하라!” <https://dacon.io/competitions/official/236488/overview/description>